

HW5: executed notebook

HW5: Supervised fine-tuning and ability trade-offs

Completed experiment on **Qwen/Qwen2.5-0.5B-Instruct** using LoRA. This notebook compares the unchanged base model, a narrow machine-learning-only ablation, and mixed-domain SFT. All reported predictions are produced by the models. No public evaluation answers or hidden-test questions are used for training.

Run: In Colab select a GPU runtime, upload this notebook, `requirements.txt`, `hw5_sample_eval.csv`, and `kaggle_test.csv` to the working directory, then Run all. Downloads require internet. Local users should install the requirements in a fresh environment with CUDA-enabled PyTorch. The dataset/model revisions below fix the source versions; exact GPU floating-point results can vary across hardware.

The starter explicitly trains on the MMLU test split. We follow that convention for eight subjects and evaluate on their separate dev/validation splits; these are **not standard MMLU benchmark scores**. Training questions matching either course CSV are removed. Public course evaluation overlaps the hidden CSV, so it is only a descriptive check, not a hyperparameter-selection target.

GenAI acknowledgement: OpenAI Codex assisted with implementation, paper explanations, debugging, and report preparation. Experiment outputs were computed by the recorded runs.

Code cell 1 | execution 1

```
import sys, subprocess
if "google.colab" in sys.modules:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", "-r", "requirements.txt"])
```

Code cell 2 | execution 2

```
"""Reproducible HW5 SFT helpers. The notebook contains the same source."""
from pathlib import Path
import gc
import hashlib
import json
import random
import re
import unicodedata
import numpy as np
import pandas as pd
import torch
from datasets import Dataset, load_dataset
from transformers import AutoModelForCausalLM, AutoTokenizer, set_seed
from peft import LoraConfig, PeftModel
from trl import SFTConfig, SFTTrainer
from huggingface_hub import snapshot_download

MODEL_NAME = "Qwen/Qwen2.5-0.5B-Instruct"
SEED = 189
LETTERS = "ABCDE"
SPECIALIZED = ["machine_learning", "college_computer_science", "college_mathematics", "high_school_statistics"]
GENERAL = ["high_school_world_history", "high_school_geography", "nutrition", "elementary_mathematics"]
SUBJECTS = SPECIALIZED + GENERAL
```

Code cell 3 | execution 3

```
def fingerprint(question):
    text = unicodedata.normalize("NFKC", str(question)).casefold()
    return re.sub(r"\W+", "", text)
```

Code cell 4 | execution 4

```
def read_csv(path, labeled=True):
    frame = pd.read_csv(path, keep_default_na=False)
    required = {"id", "question", "A", "B", "C", "D", "E"}
    if labeled:
        required.add("answer")
    if not required.issubset(frame.columns):
        raise ValueError(f"Missing columns: {required - set(frame.columns)}")
    if frame.id.duplicated().any():
        raise ValueError("Duplicate IDs")
    if labeled and not frame.answer.isin(list(LETTERS)).all():
        raise ValueError("Invalid answer labels")
    return frame
```

Code cell 5 | execution 5

```
def row_record(row, group="cs189"):
    choices = [str(row[c]).strip() for c in LETTERS if str(row[c]).strip()]
    if len(choices) < 2:
        raise ValueError("A multiple-choice question needs at least two options")
    # The supplied CSVs have contiguous, nonempty choices; reject ambiguous gaps.
    if any(not str(row[c]).strip() for c in LETTERS[:len(choices)]):
        raise ValueError("Choice labels must be contiguous")
    return {"id": str(row["id"]), "question": str(row["question"]), "choices": choices,
            "answer": row.get("answer", ""), "group": group, "source": "course CSV"}
```

Code cell 6 | execution 6

```
def build_prompt(record):
    options = "\n".join(f"{LETTERS[i]}. {choice}" for i, choice in enumerate(record["choices"]))
    return ("Choose exactly one correct option from the choices provided. "
            "Return only the option letter inside a LaTeX box.\n\n"
            + record["question"].strip() + "\n\n" + options)
```

Code cell 7 | execution 7

```
def parse_choice_from_boxed(text):
    if not isinstance(text, str):
        return None
    boxed = re.findall(r"\\boxed\\s*\\{\\s*([A-E])\\s*\\}", text, flags=re.I)
    if boxed:
        return boxed[-1].upper()
    exact = re.fullmatch(r"\\s*([A-E])[.]?\\s*", text, flags=re.I)
    return exact.group(1).upper() if exact else None
```

Code cell 8 | execution 8

```
def prepare_data(data_dir, revisions, eval_csv, test_csv):
    data_dir = Path(data_dir)
    data_dir.mkdir(exist_ok=True)
    public = [row_record(row) for row in read_csv(eval_csv).to_dict("records")]
    hidden = [row_record(row, "hidden") for row in read_csv(test_csv, False).to_dict("records")]
    forbidden = {fingerprint(r["question"]) for r in public + hidden}
    records = {"train": [], "validation": []}
    excluded = 0
    for subject in SUBJECTS:
        for split in ("test", "validation", "dev"):
            path = data_dir / f"{subject}_{split}.jsonl"
            if path.exists():
                rows = [json.loads(line) for line in path.read_text(encoding="utf-8").splitlines()]
            else:
                ds = load_dataset("cais/mmlu", subject, split=split, revision=revisions["cais/mmlu"])
                ds.to_json(str(path))
                rows = list(ds)
            destination = "train" if split == "test" else "validation"
            for index, row in enumerate(rows):
                record = {"id": f"mmlu_{subject}_{split}_{index}", "question": row["question"],
                        "choices": list(row["choices"]), "answer": LETTERS[int(row["answer"])],
                        "group": "specialized" if subject in SPECIALIZED else "general",
                        "subject": subject, "source": f"cais/mmlu:{subject}:{split}"}
                if fingerprint(record["question"]) in forbidden:
                    excluded += 1
                else:
                    records[destination].append(record)
    validation_keys = {fingerprint(r["question"]) for r in records["validation"]}
    seen = set()
    cleaned = []
    rng = random.Random(SEED)
    for subject in SUBJECTS:
        candidates = [r for r in records["train"] if r["subject"] == subject]
        rng.shuffle(candidates)
        for r in candidates[:120]:
            key = fingerprint(r["question"])
            if key not in seen and key not in validation_keys:
                cleaned.append(r)
                seen.add(key)
    rng.shuffle(cleaned)
    assert not seen.intersection(forbidden | validation_keys)
    manifest = {"model": MODEL_NAME, "revisions": revisions, "seed": SEED,
               "train_count": len(cleaned), "validation_count": len(records["validation"]),
               "public_count": len(public), "hidden_count": len(hidden),
               "excluded_test_overlap": excluded,
               "public_hidden_overlap": len({fingerprint(r["question"]) for r in public} &
                                           {fingerprint(r["question"]) for r in hidden}),
               "train_counts": pd.Series([r["subject"] for r in cleaned]).value_counts().to_dict(),
               "note": "MMLU test split is explicitly training data, as in the starter; dev/validation are held
out."}
    return cleaned, records["validation"], public, hidden, manifest
```

Code cell 9 | execution 9

```
def training_dataset(records, tokenizer):
    formatted = []
    rng = random.Random(SEED)
    for record in records:
        order = list(range(len(record["choices"])))
        rng.shuffle(order)
        original_answer = LETTERS.index(record["answer"])
        permuted = dict(record, choices=[record["choices"][i] for i in order])
        answer = LETTERS[order.index(original_answer)]
        prompt = tokenizer.apply_chat_template([{"role": "user", "content": build_prompt(permuted)}],
                                              tokenize=False, add_generation_prompt=True)
        formatted.append({"prompt": prompt, "completion": f"\\boxed{{{answer}}}" + tokenizer.eos_token})
    return Dataset.from_list(formatted)
```

Code cell 10 | execution 10

```
def load_base(revision):
    set_seed(SEED)
    snapshot = snapshot_download(MODEL_NAME, revision=revision,
                                allow_patterns=["*.json", "*.safetensors", "*.txt"])
    tokenizer = AutoTokenizer.from_pretrained(snapshot)
    tokenizer.pad_token = tokenizer.eos_token
    tokenizer.padding_side = "left"
    dtype = torch.bfloat16 if torch.cuda.is_available() and torch.cuda.is_bf16_supported() else torch.float32
    model = AutoModelForCausalLM.from_pretrained(snapshot, dtype=dtype,
                                                attn_implementation="sdpa")
    model.to("cuda" if torch.cuda.is_available() else "cpu")
    model.eval()
    return model, tokenizer
```

Code cell 11 | execution 11

```
def predict_choices(model, tokenizer, records, batch_size=4):
    """Constrained one-token decoding after a forced boxed-answer prefix.

    Only supplied option letters are eligible. This is used identically for base,
    ablation, final model and hidden test; no labels enter the scoring path.
    """
    model.eval()
    device = next(model.parameters()).device
    letter_ids = [tokenizer.encode(letter, add_special_tokens=False) for letter in LETTERS]
    if not all(len(ids) == 1 for ids in letter_ids):
        raise ValueError("This decoder requires single-token answer letters")
    token_ids = [ids[0] for ids in letter_ids]
    rows = []
    tokenizer.padding_side = "left"
    for start in range(0, len(records), batch_size):
        batch = records[start:start+batch_size]
        prompts = [tokenizer.apply_chat_template([{"role": "user", "content": build_prompt(r)}],
                                                tokenize=False, add_generation_prompt=True) + "\\boxed{" for r in batch]
        inputs = tokenizer(prompts, padding=True, return_tensors="pt").to(device)
        # Match Trainer/Accelerate's mixed-precision context after adapter reload.
        with torch.inference_mode(), torch.autocast(
            device_type=device.type, dtype=torch.bfloat16,
            enabled=device.type == "cuda" and next(model.parameters()).dtype == torch.bfloat16):
            logits = model(**inputs, use_cache=False, logits_to_keep=1).logits[:, -1, token_ids].float()
        for i, r in enumerate(batch):
            scores = logits[i, :len(r["choices"])].cpu()
            prediction = LETTERS[int(scores.argmax())]
            rows.append({**r, "prediction": prediction, "output": f"\\boxed{{{prediction}}}",
                        "correct": prediction == r.get("answer", ""),
                        "choice_probabilities": torch.softmax(scores, dim=0).tolist()})
    return pd.DataFrame(rows)
```

Code cell 12 | execution 12

```
def summarize(frame):
    return {group: {"correct": int(g.correct.sum()), "n": len(g), "accuracy": float(g.correct.mean())}
            for group, g in frame.groupby("group")}
```

Code cell 13 | execution 13

```
def train_adapter(model, tokenizer, records, output_dir, epochs=2, learning_rate=1e-4, max_steps=-1):
    set_seed(SEED)
    tokenizer.padding_side = "right"
    model.config.use_cache = False
    config = SFTConfig(output_dir=str(output_dir), num_train_epochs=epochs, max_steps=max_steps,
                       per_device_train_batch_size=1, gradient_accumulation_steps=16,
                       learning_rate=learning_rate, warmup_ratio=0.1, lr_scheduler_type="cosine",
                       weight_decay=0.01, optim="adamw_torch", max_grad_norm=1.0,
                       logging_steps=5, save_strategy="no", report_to="none", seed=SEED,
                       data_seed=SEED, max_length=1024, packing=False, completion_only_loss=True,
                       bf16=torch.cuda.is_available() and torch.cuda.is_bf16_supported(),
                       fp16=False, gradient_checkpointing=True,
                       gradient_checkpointing_kwargs={"use_reentrant": False},
                       dataloader_num_workers=0)
    lora = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05, target_modules="all-linear",
                     bias="none", task_type="CAUSAL_LM")
    trainer = SFTTrainer(model=model, args=config, train_dataset=training_dataset(records, tokenizer),
                        processing_class=tokenizer, peft_config=lora)
    result = trainer.train()
    trainer.save_model(str(output_dir))
    tokenizer.save_pretrained(str(output_dir))
    history = trainer.state.log_history
    (Path(output_dir)/"loss_history.json").write_text(json.dumps(history, indent=2))
    model = trainer.model
    model.eval()
    return model, history, result.metrics
```

Code cell 14 | execution 14

```
def compare_examples(base, final, count=2):
    before = base.set_index("id")
    after = final.set_index("id")
    assert before.index.equals(after.index)
    general = before.group == "general"
    result = {}
    for name, mask in [("forgetting", general & before.correct & ~after.correct),
                      ("gains", general & ~before.correct & after.correct)]:
        selected = sorted(list(before.index[mask]), key=lambda idx:
                              len(before.loc[idx, 'question']) + sum(map(len, before.loc[idx, 'choices']))):count]
        result[name] = [{"id": idx, "question": before.loc[idx, "question"],
                        "choices": before.loc[idx, "choices"], "answer": before.loc[idx, "answer"],
                        "base_output": before.loc[idx, "output"], "final_output": after.loc[idx, "output"],
                        "source": before.loc[idx, "source"]} for idx in selected]

    return result
```

Code cell 15 | execution 15

```
def save_submission(predictions, test_csv, output_path):
    ids = read_csv(test_csv, False).id.astype(str).tolist()
    submission = predictions[["id", "prediction"]].copy()
    if submission.id.astype(str).tolist() != ids or submission.id.duplicated().any():
        raise ValueError("Prediction IDs must match the test file exactly and in order")
    if not submission.prediction.isin(list(LETTERS)).all():
        raise ValueError("Invalid prediction")
    submission.to_csv(output_path, index=False)
    return submission
```

Data and reproducibility checks

Answer choices are permuted once with seed 189 during training, with labels remapped. Only completion tokens contribute to SFT loss. Evaluation uses identical constrained letter decoding for every model: append `\boxed{` to the assistant prefix and choose the highest-logit valid answer letter. Reported boxed outputs are formatted constrained predictions, not free-form explanations.

Code cell 17 | execution 16

```
import os, platform, importlib.metadata
import matplotlib.pyplot as plt
from IPython.display import display
WORK = Path.cwd()
RESULTS = WORK / "results"
RESULTS.mkdir(exist_ok=True)
REVISIONS = {"model": "7ae557604adf67be50417f59c2c2f167def9a775",
             "cais/mmlu": "c30699e8356da336a370243923dbaf21066bb9fe"}
train, heldout, public, hidden, manifest = prepare_data(
    WORK / "data", REVISIONS, WORK / "hw5_sample_eval.csv", WORK / "kaggle_test.csv")
evaluation = heldout + public
manifest["environment"] = {p: importlib.metadata.version(p) for p in
    ["torch", "transformers", "trl", "peft", "datasets"]}
manifest["gpu"] = torch.cuda.get_device_name(0) if torch.cuda.is_available() else "CPU"
print(json.dumps(manifest, indent=2))
assert parse_choice_from_boxed(r"Answer: \boxed{C}") == "C"
assert parse_choice_from_boxed("Because A might be right") is None
assert len({fingerprint(r["question"]) for r in train}) == len(train)
{
    "model": "Qwen/Qwen2.5-0.5B-Instruct",
    "revisions": {
        "model": "7ae557604adf67be50417f59c2c2f167def9a775",
        "cais/mmlu": "c30699e8356da336a370243923dbaf21066bb9fe"
    },
    "seed": 189,
    "train_count": 907,
    "validation_count": 217,
    "public_count": 25,
    "hidden_count": 169,
    "excluded_test_overlap": 7,
    "public_hidden_overlap": 21,
    "train_counts": {
        "high_school_statistics": 120,
        "high_school_geography": 120,
        "high_school_world_history": 120,
        "elementary_mathematics": 120,
        "nutrition": 119,
        "machine_learning": 108,
        "college_computer_science": 100,
        "college_mathematics": 100
    },
    "note": "MMLU test split is explicitly training data, as in the starter; dev/validation are held out.",
    "environment": {
        "torch": "2.6.0+cu124",
        "transformers": "4.57.3",
        "trl": "0.24.0",
        "peft": "0.17.1",
        "datasets": "3.6.0"
    },
    "gpu": "NVIDIA GeForce RTX 4060 Laptop GPU"
}
```

Base model

The general held-out questions cover history, geography, nutrition, and elementary mathematics. The specialized group covers machine learning, computer science, college mathematics, and statistics.

Code cell 19 | execution 17

```
model, tokenizer = load_base(REVISIONS["model"])
lengths = [len(tokenizer(r["prompt"] + r["completion"]))["input_ids"]]
    for r in training_dataset(train, tokenizer)]
assert max(lengths) <= 1024, "Increase max_length before training; never truncate away the answer"
print("Maximum training token length:", max(lengths))
base_predictions = predict_choices(model, tokenizer, evaluation)
base_predictions.to_json(RESULTS / "base_predictions.json", orient="records", indent=2)
metrics = {"base": summarize(base_predictions)}
print(json.dumps(metrics, indent=2))
del model
gc.collect()
torch.cuda.empty_cache()
```

```

Maximum training token length: 800
{
  "base": {
    "cs189": {
      "correct": 6,
      "n": 25,
      "accuracy": 0.24
    },
    "general": {
      "correct": 73,
      "n": 142,
      "accuracy": 0.5140845070422535
    },
    "specialized": {
      "correct": 30,
      "n": 75,
      "accuracy": 0.4
    }
  }
}

```

Ablation: narrow SFT

Train only on machine-learning questions for 30 optimizer steps at learning rate 0.0003. This tests whether repeatedly fitting a small domain dataset is sufficient to improve the broader task while retaining general knowledge.

Code cell 21 | execution 18

```

model, tokenizer = load_base(REVISIONS["model"])
narrow_data = [r for r in train if r["subject"] == "machine_learning"]
model, narrow_history, narrow_training = train_adapter(
    model, tokenizer, narrow_data, WORK / "models" / "narrow", learning_rate=3e-4, max_steps=30)
narrow_predictions = predict_choices(model, tokenizer, evaluation)
narrow_predictions.to_json(RESULTS / "narrow_predictions.json", orient="records", indent=2)
metrics["narrow"] = summarize(narrow_predictions)
print(json.dumps(metrics["narrow"], indent=2))
del model
gc.collect()
torch.cuda.empty_cache()
Adding EOS to train dataset: 0%|          | 0/108 [00:00<?, ? examples/s]
Tokenizing train dataset: 0%|          | 0/108 [00:00<?, ? examples/s]
Truncating train dataset: 0%|          | 0/108 [00:00<?, ? examples/s]
The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation config. The model
config and generation config were aligned accordingly, being updated with the tokenizer's values. Updated
tokens: {'bos_token_id': None, 'pad_token_id': 151645}.
[30/30 02:31, Epoch 4/5]
Step | Training Loss |
5 | 0.753800 |
10 | 0.203400 |
15 | 0.198800 |
20 | 0.142800 |
25 | 0.097500 |
30 | 0.076700 |
{
  "cs189": {
    "correct": 12,
    "n": 25,
    "accuracy": 0.48
  },
  "general": {
    "correct": 71,
    "n": 142,
    "accuracy": 0.5
  },
  "specialized": {
    "correct": 26,
    "n": 75,
    "accuracy": 0.3466666666666667
  }
}

```

Final method: mixed SFT

Start again from the unchanged base model, then train on a balanced cap of 120 questions per subject for two epochs. LoRA uses rank 16, alpha 32, dropout 0.05 on all linear layers; AdamW uses peak learning rate 0.0001, weight decay 0.01, cosine decay, 10% warmup, effective batch 16 and gradient clipping at 1. This is a one-stage mixed-data experiment motivated by replay and data-composition work; it is not a reproduction of the paper's two-stage DMT.

Code cell 23 | execution 19

```
model, tokenizer = load_base(REVISIONS["model"])
model, final_history, final_training = train_adapter(
    model, tokenizer, train, WORK / "models" / "final", epochs=2, learning_rate=1e-4)
final_predictions = predict_choices(model, tokenizer, evaluation)
final_predictions.to_json(RESULTS / "final_predictions.json", orient="records", indent=2)
metrics["mixed"] = summarize(final_predictions)
(RESULTS / "metrics.json").write_text(json.dumps(metrics, indent=2))
(RESULTS / "manifest.json").write_text(json.dumps(manifest, indent=2))
histories = {"narrow": narrow_history, "mixed": final_history}
(RESULTS / "training_history.json").write_text(json.dumps(histories, indent=2))
(RESULTS / "training_metrics.json").write_text(json.dumps(
    {"narrow": narrow_training, "mixed": final_training}, indent=2))
table = pd.DataFrame([{"model": name, "group": group, **values}
    for name, groups in metrics.items() for group, values in groups.items()])

display(table)
fig, ax = plt.subplots(figsize=(7, 3))
for name, history in histories.items():
    points = [r for r in history if "loss" in r]
    ax.plot([r["step"] for r in points], [r["loss"] for r in points], marker=".", label=name)
ax.set(xlabel="Optimizer step", ylabel="Completion cross-entropy", title="Actual SFT training loss")
ax.legend(); fig.tight_layout()
fig.savefig(RESULTS / "loss_curves.png", dpi=160)
plt.show()
Adding EOS to train dataset: 0%|          | 0/907 [00:00<?, ? examples/s]
Tokenizing train dataset: 0%|          | 0/907 [00:00<?, ? examples/s]
Truncating train dataset: 0%|          | 0/907 [00:00<?, ? examples/s]
The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation config. The model
config and generation config were aligned accordingly, being updated with the tokenizer's values. Updated
tokens: {'bos_token_id': None, 'pad_token_id': 151645}.
[114/114 10:53, Epoch 2/2]
Step | Training Loss |
5 | 1.246000 |
10 | 0.313400 |
15 | 0.231200 |
20 | 0.219000 |
25 | 0.192600 |
30 | 0.215100 |
35 | 0.205000 |
40 | 0.211700 |
45 | 0.201300 |
50 | 0.198700 |
55 | 0.210100 |
60 | 0.159400 |
65 | 0.189100 |
70 | 0.159100 |
75 | 0.133500 |
80 | 0.151400 |
85 | 0.152200 |
90 | 0.153700 |
95 | 0.154100 |
100 | 0.138800 |
105 | 0.139700 |
110 | 0.153400 |
| model | group | correct | n | accuracy |
0 | base | cs189 | 6 | 25 | 0.240000 |
1 | base | general | 73 | 142 | 0.514085 |
2 | base | specialized | 30 | 75 | 0.400000 |
3 | narrow | cs189 | 12 | 25 | 0.480000 |
4 | narrow | general | 71 | 142 | 0.500000 |
5 | narrow | specialized | 26 | 75 | 0.346667 |
6 | mixed | cs189 | 10 | 25 | 0.400000 |
7 | mixed | general | 72 | 142 | 0.507042 |
8 | mixed | specialized | 28 | 75 | 0.373333 |
```




Observed forgetting and gains

Select the two shortest changed examples of each type from the held-out general subjects, solely to keep the report readable. These are individual errors and improvements under the same decoder; they do not by themselves establish population-wide catastrophic forgetting or causal transfer.

Code cell 25 | execution 20

```
examples = compare_examples(base_predictions, final_predictions)
assert all(len(rows) >= 2 for rows in examples.values()), "Need at least two real examples of each type"
(RESULTS / "examples.json").write_text(json.dumps(examples, indent=2))
for category, rows in examples.items():
    print("\n" + category.upper())
    for row in rows:
        print(build_prompt(row))
        print("Base:", row["base_output"], "Final:", row["final_output"], "Gold:", row["answer"])
```

FORGETTING
Choose exactly one correct option from the choices provided. Return only the option letter inside a LaTeX box.

Solve for y . $y - 2 + 3y = 10$

A. 2
B. 3
C. 4
D. 5
Base: $\boxed{B}$ Final: $\boxed{C}$ Gold: B
Choose exactly one correct option from the choices provided. Return only the option letter inside a LaTeX box.

Solve the equation $29 = x - 17$.

A. 12
B. -12
C. -46
D. 46
Base: $\boxed{D}$ Final: $\boxed{C}$ Gold: D

GAINS
Choose exactly one correct option from the choices provided. Return only the option letter inside a LaTeX box.

If 50% of a number is 20, what is 75% of the number?

A. 8
B. 15
C. 30
D. 45
Base: $\boxed{B}$ Final: $\boxed{C}$ Gold: C
Choose exactly one correct option from the choices provided. Return only the option letter inside a LaTeX box.
The expression $-105 + (-14) + 34$ simplifies to which of the following?

A. -57
B. -75
C. -85
D. 143
Base: \boxed{A} Final: \boxed{C} Gold: C

Hidden-test predictions and submission checks

Every row is inferred with the trained mixed adapter. No hidden answers are available. Upload `submission.csv` to the course competition, then supply your real Kaggle username and score screenshot for the writeup. A local CSV is not proof of Kaggle submission.

Code cell 27 | execution 21

```
hidden_predictions = predict_choices(model, tokenizer, hidden)
submission = save_submission(hidden_predictions, WORK / "kaggle_test.csv", WORK / "submission.csv")
assert submission.columns.tolist() == ["id", "prediction"]
assert len(submission) == len(hidden)
print("Saved submission.csv:", len(submission), "unique IDs")
print(submission.prediction.value_counts().sort_index())
display(submission.head())
print("Kaggle score: not available until an authenticated submission is completed.")
Saved submission.csv: 169 unique IDs
prediction
A    39
B    40
C    46
D    29
E    15
Name: count, dtype: int64
| id | prediction |
0 | test_00000 | A |
1 | test_00001 | E |
2 | test_00002 | B |
3 | test_00003 | B |
4 | test_00004 | A |
Kaggle score: not available until an authenticated submission is completed.
```

Sources

- Base model: <https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct>
- MCQ data and subject descriptions: <https://huggingface.co/datasets/cais/mmlu>
- SFTTrainer and completion-only loss: https://huggingface.co/docs/trl/v0.24.0/sft_trainer
- LoRA: <https://arxiv.org/abs/2106.09685>
- Data composition: <https://arxiv.org/abs/2310.05492>

The narrow and mixed runs differ in both data coverage and optimization budget, so their comparison is an exploratory ablation, not a controlled estimate of the isolated effect of mixing. The public CS189 set has only 25 questions; one answer changes accuracy by four percentage points.

Reload verification

The inference helper explicitly uses `bfloat16` autocast on the tested CUDA model. This matches the precision context applied by Trainer/Accelerate during training and avoids a change in LoRA arithmetic after reloading an adapter. After this compatibility fix, all **895 saved predictions** were rechecked: 242 each for the base, narrow and mixed models, plus 169 hidden predictions; every predicted letter matched. `validate_hw5.py --reload-model` also checks a saved-adapter inference batch. The metrics and plots above are unchanged by this fix.

Kaggle result received after this run

The user subsequently submitted `submission.csv`. Their screenshots show account **Shiyu63**, Public Score **0.40476**, Private Score **0.35294**, and status **Complete (after deadline)**. Earlier cell outputs describe the state at execution time, before this submission. The late-submission score does not confirm course-credit eligibility. The score image still needs to be saved as `results/kaggle_score.png` for embedding in the report.

Appendix: artifact and validation scripts

export_deliverables.py

```
"""Export the written answers, full executed notebook and two-page report."""
from pathlib import Path
import base64
import io
import json
import re
import textwrap
from html.parser import HTMLParser
from xml.sax.saxutils import escape
import nbformat
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.lib import colors
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.enums import TA_CENTER
from reportlab.platypus import (SimpleDocTemplate, Paragraph, Spacer, Preformatted,
                                Image, PageBreak, Table, TableStyle, KeepTogether)

HERE = Path(__file__).resolve().parent
STYLES = getSampleStyleSheet()
for name in ['Heading1', 'Heading2', 'Heading3']:
    STYLES[name].keepWithNext=True
STYLES.add(ParagraphStyle('BodySmall', parent=STYLES['BodyText'], fontSize=9, leading=12, spaceAfter=6))
STYLES.add(ParagraphStyle('CodeSmall', fontName='Courier', fontSize=7, leading=9))
for candidate in ['C:/Windows/Fonts/consola.ttf', '/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf']:
    if Path(candidate).exists():
        pdfmetrics.registerFont(TTFont('CodeUnicode', candidate))
        STYLES['CodeSmall'].fontName='CodeUnicode'
        break

def inline(text):
    text = escape(text)
    text = re.sub(r'\\([^\]]+)\|\\((https?:/[^\]]+)\)', r'<link href="\2" color="#17578a">\1</link>', text)
    text = re.sub(r'\\*\\*([^\]]+)\\*\\*', r'<b>\1</b>', text)
    text = re.sub(r'\\(?![^\]]+)\\*([^\]]+)\\*(?!\\*)', r'<i>\1</i>', text)
    return re.sub(r'`([^\`]+)`', r'<font name="Courier">\1</font>', text)

def markdown_blocks(text, small=False):
    result=[]
    text=re.sub(r'(?m)^(#{1,6} [^\n]+)\n(?:=|_){1,6}\n',r'\1\n\n',text)
    for block in re.split(r'\n\s*\n', text.strip()):
        if block.startswith('#'):
            depth=len(block)-len(block.lstrip('#'))
            style=STYLES['Title' if depth==1 else 'Heading2' if depth==2 else 'Heading3']
            result.append(Paragraph(inline(block.lstrip('#')), style))
        else:
            result.append(Paragraph(inline(block).replace('\n','<br/>'), STYLES['BodySmall' if small else 'BodyText']))
        if not block.startswith('#'):
            result.append(Spacer(1,4))
    return result

def footer(canvas, doc):
    canvas.setFont('Helvetica',8)
    canvas.setFillColor(colors.grey)
    canvas.drawString(42,25,'CS189 HW5 | Reproducible SFT experiment')
    canvas.drawRightString(570,25,str(doc.page))

def build_pdf(name, story, **kwargs):
    doc=SimpleDocTemplate(str(HERE/name), pagesize=(612,792), rightMargin=42,leftMargin=42,
                        topMargin=35,bottomMargin=38,**kwargs)
    doc.build(story,onFirstPage=footer,onLaterPages=footer)

def code_lines(text, width=112):
    # Preserve every code/output character, wrapping long lines instead of clipping.
    lines=[]
    for line in text.expandtabs(4).splitlines():
```

```

        lines.extend(textwrap.wrap(line, width=width, replace_whitespace=False,
                                   drop_whitespace=False, break_long_words=True,
                                   break_on_hyphens=False) or [''])
    return '\n'.join(lines)

class VisibleHTML(HTMLParser):
    """Preserve visible notebook HTML output, including Trainer's loss table."""
    def __init__(self):
        super().__init__(); self.parts=[]; self.hidden=False
    def handle_starttag(self,tag,attrs):
        if tag in ('style','script'):self.hidden=True
    def handle_endtag(self,tag):
        if tag in ('style','script'):self.hidden=False
        if tag in ('td','th'):self.parts.append(' | ')
        if tag in ('tr','p','div','br'):self.parts.append('\n')
    def handle_data(self,data):
        if not self.hidden:
            self.parts.append(re.sub(r'\s+', ' ',data))
    def text(self):
        return '\n'.join(line.strip() for line in ''.join(self.parts).splitlines() if line.strip())

def export_notebook():
    nb=nbformat.read(HERE/'finetuning_tutorial.ipynb',as_version=4)
    story=[Paragraph('HW5: executed notebook',STYLES['Title'])]
    for index,cell in enumerate(nb.cells):
        if cell.cell_type=='markdown':
            story.extend(markdown_blocks(cell.source,small=True))
        elif cell.cell_type=='code':
            story.append(Paragraph(f'Code cell {index} | execution {cell.execution_count}',STYLES['Heading3']))
            story.append(Preformatted(code_lines(cell.source),STYLES['CodeSmall']))
            for output in cell.get('outputs',[]):
                if output.output_type=='error':
                    raise ValueError(f'Notebook contains an error in cell {index}')
                data=output.get('data',{})
                if 'image/png' in data:
                    pic=Image(io.BytesIO(base64.b64decode(data['image/png'])))
                    ratio=min(528/pic.imageWidth,300/pic.imageHeight)
                    pic.drawWidth=pic.imageWidth*ratio;pic.drawHeight=pic.imageHeight*ratio
                    story.append(pic)
                elif 'text/html' in data:
                    parser=VisibleHTML();parser.feed(data['text/html'])
                    story.append(Preformatted(code_lines(parser.text()),STYLES['CodeSmall']))
                elif output.output_type=='stream' or 'text/plain' in data:
                    value=output.get('text',data.get('text/plain',''))
                    value=re.sub(r'\x1b\[0-9;]*[A-Za-z]','',value).replace('\r','\n')
                    story.append(Preformatted(code_lines(value),STYLES['CodeSmall']))
            story.append(Spacer(1,10))
    story.append(PageBreak())
    story.append(Paragraph('Appendix: artifact and validation scripts',STYLES['Title']))
    for filename in ['export_deliverables.py','validate_hw5.py','package_submission.py']:
        story.append(Paragraph(filename,STYLES['Heading2']))
        lines=code_lines((HERE/filename).read_text(encoding='utf-8')).splitlines()
        for start in range(0,len(lines),40):
            story.append(Preformatted('\n'.join(lines[start:start+40]),STYLES['CodeSmall']))
    build_pdf('finetuning_tutorial.pdf',story)

def export_report():
    results=HERE/'results'
    metrics=json.loads((results/'metrics.json').read_text())
    manifest=json.loads((results/'manifest.json').read_text())
    examples=json.loads((results/'examples.json').read_text())
    story=[Paragraph('HW5 SFT experiment report',STYLES['Title'])]
    method=(f'I fine-tuned Qwen2.5-0.5B-Instruct with LoRA on {manifest["train_count"]} four-choice '
            'MMLU questions from machine learning, computer science, college mathematics, statistics, history, '
            'geography, nutrition, and elementary mathematics, using at most 120 training questions per subject [1,2]. '
            'As in the starter, MMLU test splits supply training data; dev/validation splits are held out, and '
            'questions '
            'matching either course CSV are removed before training. '
            'The final run uses two epochs, LoRA rank 16/alpha 32/dropout 0.05 on all linear layers, AdamW at peak '
            'learning rate 1e-4, weight decay 0.01, cosine decay, 10% warmup, batch 1 with 16-step accumulation, '
            'gradient clipping 1, bfloat16 and seed 189 [3]. '
            'SFTTrainer minimizes completion-only cross-entropy on boxed answer letters after deterministic choice '
            'permutation; base and tuned models use the same constrained letter decoding [4]. '
            'Mixing general and specialized examples is motivated by data-composition work [5], but this one-stage ')

```



```

refs+='[2] <link href="https://huggingface.co/datasets/cais/mmlu">cais/mmlu</link>. '
refs+='[3] <link href="https://arxiv.org/abs/2106.09685">Hu et al., LoRA</link>. '
refs+='[4] <link href="https://huggingface.co/docs/trl/v0.24.0/sft_trainer">TRL SFTTrainer</link>. '
refs+='[5] <link href="https://arxiv.org/abs/2310.05492">Dong et al., SFT data composition</link>. '
refs+='OpenAI Codex assisted with code, debugging and writing; results come from actual model runs.'
story.append(Paragraph(refs,STYLES['BodySmall']))
build_pdf('hw5_writeup.pdf',story)

if __name__=='__main__':
    import sys
    build_pdf('hw5_paper_answers.pdf',markdown_blocks((HERE/'hw5_paper_answers.md').read_text(encoding='utf-8')))
    if '--paper-only' not in sys.argv:
        export_notebook()
        export_report()
    print('PDF exports complete')

```

validate_hw5.py

```

"""Validate data isolation, saved predictions, notebook/source agreement and artifacts."""
from pathlib import Path
import ast
import json
import tempfile
import sys
import nbformat
import pandas as pd
from sft_pipeline import (prepare_data, fingerprint, read_csv, build_prompt,
                          parse_choice_from_boxed, summarize, save_submission)

HERE=Path(__file__).resolve().parent

def main():
    revisions=json.loads((HERE/'source_revisions.json').read_text())
    train,heldout,public,hidden,manifest=prepare_data(
        HERE/'data',revisions,HERE/'hw5_sample_eval.csv',HERE/'kaggle_test.csv')
    assert not ({fingerprint(r['question']) for r in train} &
                {fingerprint(r['question']) for r in heldout+public+hidden})
    assert build_prompt(dict(public[0],answer='A'))==build_prompt(dict(public[0],answer='E'))
    for text,expected in [(r'\boxed{C}','C'),(r'\boxed{A} then \boxed{d}','D'),
                          ('B.','B'),('Because A appears in this sentence',None),('','None')]:
        assert parse_choice_from_boxed(text)==expected
    metrics=json.loads((HERE/'results/metrics.json').read_text())
    frames={}
    for file,key in [(('base','base'),('narrow','narrow'),('final','mixed'))]:
        frame=pd.read_json(HERE/f'results/{file}_predictions.json')
        assert frame.id.tolist()==[r['id'] for r in heldout+public]
        assert (frame.correct==(frame.prediction==frame.answer)).all()
        assert summarize(frame)==metrics[key]
        frames[file]=frame.set_index('id')
    examples=json.loads((HERE/'results/examples.json').read_text())
    for category,rows in examples.items():
        assert len(rows)==2
        for r in rows:
            base,final=(frames[name].loc[r['id']] for name in ['base','final'])
            assert base['group']=='general'
            assert r['base_output']==base.output and r['final_output']==final.output
            assert bool(base.correct)==(category=='forgetting')
            assert bool(final.correct)==(category=='gains')
    submission=pd.read_csv(HERE/'submission.csv',dtype=str)
    assert submission.columns.tolist()==['id','prediction']
    with tempfile.TemporaryDirectory() as temp:
        save_submission(submission,HERE/'kaggle_test.csv',Path(temp)/'valid.csv')
        invalid=submission.copy();invalid.loc[0,'prediction']='F'
        try:save_submission(invalid,HERE/'kaggle_test.csv',Path(temp)/'invalid.csv')
        except ValueError:pass
        else:raise AssertionError('Invalid label was accepted')
    nb=nbformat.read(HERE/'finetuning_tutorial.ipynb',as_version=4)
    code=[c for c in nb.cells if c.cell_type=='code']
    assert all(c.execution_count is not None for c in code)
    assert not any(o.output_type=='error' for c in code for o in c.outputs)
    module=ast.parse((HERE/'sft_pipeline.py').read_text())
    definitions={node.name:ast.dump(node) for node in module.body if isinstance(node,ast.FunctionDef)}
    embedded={node.name:ast.dump(node) for c in code for node in ast.parse(c.source).body
              if isinstance(node,ast.FunctionDef)}
    assert definitions==embedded, 'Notebook helpers differ from support module'

```

```

for name in ['hw5_paper_answers.pdf', 'finetuning_tutorial.pdf', 'hw5_writeup.pdf']:
    assert (HERE/name).read_bytes().startswith(b'%PDF')
if '--reload-model' in sys.argv:
    from peft import PeftModel
    from sft_pipeline import load_base, predict_choices
    model, tokenizer = load_base(revisions['model'])
    model = PeftModel.from_pretrained(model, HERE/'models/final')
    replay = predict_choices(model, tokenizer, hidden[:4])
    assert replay.prediction.tolist() == submission.prediction.iloc[:4].tolist()
    print('PASS: saved adapter reload reproduces the first hidden-test batch')
print('PASS: training isolation; label-independent prompts; answer parsing; saved metrics;')
print('real forgetting/gain examples; submission IDs/labels; executed notebook; helper parity; PDFs.')

if __name__ == '__main__': main()

```

package_submission.py

```

"""Package checked code and deliverables; omit weights and download caches."""
from pathlib import Path
import hashlib
import json
import zipfile

HERE = Path(__file__).resolve().parent

def main():
    names = ['finetuning_tutorial.ipynb', 'finetuning_tutorial.pdf', 'sft_pipeline.py',
            'export_deliverables.py', 'validate_hw5.py', 'package_submission.py', 'requirements.txt',
            'source_revisions.json', 'hw5_sample_eval.csv', 'kaggle_test.csv', 'submission.csv',
            'hw5_paper_answers.md', 'hw5_paper_answers.pdf', 'hw5_writeup.pdf', 'README.md']
    files = [HERE/name for name in names] + sorted((HERE/'results').glob('*'))
    assert all(p.is_file() for p in files)
    archive = HERE/'hw5_submission.zip'
    with zipfile.ZipFile(archive, 'w', zipfile.ZIP_DEFLATED) as z:
        for path in files:
            z.write(path, path.relative_to(HERE).as_posix())
    with zipfile.ZipFile(archive) as z:
        assert z.testzip() is None
        for path in files:
            assert z.read(path.relative_to(HERE).as_posix()) == path.read_bytes()
    print(f'Created {archive.name}: {len(files)} files, {archive.stat().st_size:,} bytes')

if __name__ == '__main__': main()

```